Introduction to Wireshark

Wireshark Interface

Interface Area	Description
Menu Bar	Contains all top-level menus: File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, Help. Everything Wireshark can do is accessible from here.
Main Toolbar	Quick access buttons for Start/Stop capture, open/save files, go to a specific packet, find packets, zoom, and coloring. Hover any button to see its tooltip.
Display Filter Bar	A text field where you type Wireshark display filter expressions (e.g., tcp.flags.syn == 1). The bar turns green when a filter is valid, red when invalid.
Packet List Pane (top)	One row per captured packet. Default columns: No., Time, Source, Destination, Protocol, Length, Info. Clicking a row selects that packet for detailed inspection below. Colour-coded by protocol/condition.
Packet Detail Pane (middle)	A collapsible tree of all protocol layers for the selected packet (Frame -Ethernet-IP-TCP-Payload). Expand each layer to see every header field, its raw value, and Wireshark's human-readable interpretation.
Packet Bytes Pane (bottom)	The raw bytes of the selected packet in hex (left) and ASCII (right). Clicking a field in the Detail Pane highlights the corresponding bytes here.
Status Bar	Shows the current capture file, packet count (total/ displayed / marked), elapsed time, and current display filter status. Also shows profile name (bottom-right).

Capture Filters Examples (BPF Syntax – Applied Before Capture)

- host 192.168.1.10 – Capture traffic to or from a specific IP address.
- src host / dst host – Capture traffic only from or only to a specific host.
- net 192.168.1.0/24 – Capture traffic within a subnet.
- port 80 – Capture traffic using a specific TCP/UDP port.
- src port / dst port – Filter by source or destination port only.
- tcp | udp | icmp | arp – Capture only specific protocols.
- tcp and port 80 – Combine protocol and port conditions.
- port 80 or port 443 – Capture multiple ports.

- not arp – Exclude a protocol.

Display Filters (Applied After Capture)

- ip.addr == 192.168.1.10 – Show packets where IP appears as source or destination.
- ip.src / ip.dst – Filter by source or destination IP only.
- ip.addr == 192.168.1.0/24 – Filter by subnet using CIDR notation.
- tcp.port == 80 – Filter packets using a specific port.
- tcp.srcport / tcp.dstport – Filter by source or destination port.
- tcp | udp | icmp | dns | http | arp – Display specific protocols.
- tcp.flags.syn == 1 – Show TCP packets with SYN flag set.
- tcp.flags.syn == 1 and tcp.flags.ack == 0 – Show SYN packets only (scan detection).
- tcp.flags.syn == 1 and tcp.flags.ack == 1 – Show SYN-ACK responses (open ports).
- tcp.flags.reset == 1 – Show RST packets (closed ports).
- icmp.type == 8 / 0 – Echo Request (8) and Echo Reply (0).
- frame.len > 100 – Filter by packet size.
- http.host == "example.com" – Filter by HTTP host header.
- dns.qry.name == "example.com" – Filter DNS queries.
- eth.addr == 00:11:22:33:44:55 – Filter by MAC address.

Part A: Generating traffic with Nmap and Analysis with Wireshark

Perform the following tasks to capture traffic, apply appropriate Wireshark filters, and analyze what occurs during different scanning techniques.

Task 1: ICMP

1. Start Wireshark. Select your lab interface (e.g., eth0).
2. Generate ICMP Traffic by running:
   nmap -sn <target>
3. Apply the following display filters:
   icmp
   icmp.type == 8
   icmp.type == 0

Analyze:

1. What is the total number of ICMP packets captured?
2. How many ICMP Echo Requests were sent? (Use filter icmp.type == 8)
3. How many ICMP Echo Replies were received? (Use filter icmp.type == 0)

4. What is the TTL value in the Echo Reply packet?
5. What is the source IP address in the Echo Request?
6. What is the Identifier field value in the first Echo Request?

Task 2: TCP Connect Scan

1. In a terminal, run a standard TCP connect scan:
   nmap -sT -Pn -F <target>
2. Apply the following display filters:
   tcp.flags.syn == 1 and tcp.flags.ack == 0 (Initial SYN only)
   tcp.flags.syn == 1 and tcp.flags.ack == 1 (SYN-ACK)
   tcp.flags.ack == 1 and tcp.flags.syn == 0 and ip.src == <scanner-ip> (Final ACK of handshake)
   tcp.flags.fin == 1 || tcp.flags.reset == 1
3. Open Statistics > Flow Graph. Select 'TCP Flows'. You should see the SYN → SYN-ACK → ACK ladder diagram.

Analyze:

1. Identify one complete three-way handshake and record:
   SYN packet number
   SYN/ACK packet number
   ACK packet number
2. For the selected handshake:
   Client Initial Sequence Number
   Server Initial Sequence Number
3. How many connections were fully established? (Count completed SYN-SYN/ACK-ACK sequences)
4. After connection establishment, does Nmap close the connection using FIN or RST?
5. How many ports were open and how many were closed?

Task 3: FIN Scan

1. Generate FIN Scan Traffic by running:
   nmap -sF -Pn -F <target>
2. Apply display filters like:
   tcp.flags.fin == 1
   tcp.flags.reset == 1

Analyze:

1. How many FIN-only packets were sent by the scanner? (Use FIN filter and count packets.)
2. What is the timestamp of the first FIN packet?
3. What is the timestamp of the last FIN packet?
4. What is the total scan duration? (Last FIN – First FIN)
5. How many RST packets were received from the target?

6. How many FIN packets did NOT receive any response?

Part B: PCAP

Use Wireshark to analyze the pcap provided and answer the following questions:

1. Identify the scanning technique used.
2. Identify the IP address of the attacker.
3. Identify the IP address of the victim.
4. Determine the range of ports that were scanned.
5. Determine how many ports were closed.
6. Determine how many ports were open, and list the open port numbers.